

BOOTCAMP DE INGENIERÍA IA 2026

DE LA ESPECIFICACIÓN A LA REALIDAD CON IA

Construyendo un Mini Jira con Claude Code, React 19 y Google Stitch

Recapitulando: El Arsenal del Arquitecto IA



specs.md (PRD)

El contrato ejecutable central. Define alcance, reglas de negocio y stack tecnológico.



backlog.md (BDD)

Historias de Usuario en formato Gherkin y Edge Cases mapeados listos para testing.



architecture.mermaid

Diagramas HLD/LLD eficientes en tokens y controlables en Git.



init_db.sql

Modelado Schema-First con Mock Data realista, listo para poblar un contenedor.

¿Qué vas a poder hacer al terminar esta sesión?



Exportar con Google Stitch

Convertir un prototipo visual en un `design.md` listo para ser consumido por Claude Code.



Forjar un Agente Determinista

Configurar `CLAUDE.md` y `.claudeignore` para que la IA ejecute contratos estrictos.



Construir con Atomic Design

Ensamblar un tablero Kanban real por piezas atómicas, controlando la memoria del agente.



Implementar Optimistic UI

Usar Zustand y `useOptimistic` de React 19 para lograr 0ms de latencia percibida.

¿Qué Vamos a Construir Hoy?

01 **Workspace Base**

Vite + React 19 + Tailwind configurado con `design.md`, `specs.md` y `CLAUDE.md` activos.

02 **Estado Global**

Zustand Store (`kanbanStore.ts`) con datos mock y selectores granulares para el Kanban.

03 **UI Atómica**

Sidebar + Layout + TaskCard + KanbanBoard con estilos fieles a Stitch.

04 **Interactividad Fluida**

Movimiento de tarjetas con `useOptimistic`: respuesta visual instantánea y rollback automático.

El Objetivo de Hoy: Nuestro Mini Jira



Tablero Kanban (3 columnas)

Columnas de "Por Hacer", "En Progreso" y "Completado" con tarjetas arrastrables.



TaskCard con Datos Reales

Tarjetas con título, asignado y prioridad usando la estructura definida en `specs.md`.



Movimiento Instantáneo

Drag & Drop con Optimistic UI: el ticket se mueve a 0ms sin esperar al servidor.



Diseño Fiel al Prototipo

Colores HEX, tipografías, sombras y bordes exactos del export de Google Stitch.

¿Qué es la Ejecución Determinista?

EJECUCIÓN DETERMINISTA (SPEC-DRIVEN)

Es el enfoque arquitectónico donde la especificación (no el código) es el artefacto primario. La IA deja de 'adivinar' layouts o lógica basándose en prompts vagos; en su lugar, compila e implementa un contrato estricto predefinido, produciendo resultados predecibles y auditables cada vez.

FASE 0

CREANDO EL PROYECTO DESDE CERO

Entrevista Socrática: De Idea a Especificaciones Ejecutables

¿Qué es la Entrevista de Arquitectura Socrática?

ENTREVISTA SOCRÁTICA DE PROYECTO

Técnica donde invertimos el rol de la IA: en lugar de ejecutar órdenes, el modelo actúa como Arquitecto Senior y nos entrevista con preguntas de opción múltiple para extraer las decisiones clave de diseño. El resultado no es código — es un conjunto de artefactos ejecutables (`specs.md`, `backlog.md`) que dirigen todas las fases posteriores con precisión y sin ambigüedad.

Del Caos a las Especificaciones: El Proceso





Actividad 0: De Specs a Proyecto Real

1

Lanzar la Entrevista de Especificación

Abrir Claude Code (c`laude`) en una carpeta vacía y ejecutar el Prompt Fundacional. La IA analiza el contexto y determina qué preguntar.

2

Responder y Decidir

Evaluar cada pregunta de Claude con criterio técnico. Cada respuesta se convierte en una línea del contrato. Sin asumir, sin defaults ocultos.

3

Revisar y Aprobar specs.md

La IA genera el documento completo. Leerlo, validarlo y confirmar antes de continuar. Esto es el **contrato que ejecutará**.

4

La IA Ejecuta el Scaffolding

Con las specs aprobadas, Claude corre los comandos; crea el proyecto, instala las dependencias acordadas y genera la estructura de carpetas (atoms/, molecules/, organisms/, store/).



Prompt 0: La Entrevista Fundacional

"Analiza los archivos del proyecto (specs.md, backlog.md, design.md si existen). Identifica TODO lo que necesitas saber para especificar el frontend sin ambigüedad y formula todas tus preguntas en una sola ronda usando 'AskUserQuestion'. No asumas ningún valor por defecto. Con mis respuestas, evalúa si quedan dudas — si las hay, haz una segunda ronda con solo las preguntas pendientes. Cuando tengas certeza total, genera frontend-specs.md en Markdown con: stack y versiones, dependencias, modelo de datos, arquitectura de componentes, estructura de carpetas y reglas de negocio. No escribas código hasta que confirme el documento."

✓ Checkpoint Actividad 0: ¿Logramos esto?



Proyecto Vite Corriendo

Template React + TypeScript levantado. Servidor activo en localhost:5173 sin errores de compilación.



specs.md Generado

Documento con stack tecnológico, entidades del modelo de datos y reglas de negocio — redactado por la IA con nuestras respuestas.



backlog.md con Gherkin

Historias de usuario en formato Given/When/Then para las 3 funcionalidades core del tablero Kanban.



Decisiones Documentadas

Stack y modelo de datos elegidos por el humano. La arquitectura quedó registrada en los artefactos antes de escribir código.

FASE 1

EL PUENTE VISUAL CON GOOGLE STITCH

Traduciendo Píxeles a Tokens para Agentes CLI

Novedades en Stitch: Modo Prototipo y Voice Canvas



Rapid Prototyping (Stitch & Play)

Modo prototipo interactivo. Enlaza pantallas para definir flujos y presiona 'Play' para validar la UX antes de programar.



Voice Canvas

Edición conversacional mediante voz con Gemini Live. Ajusta el diseño hablando directamente con el lienzo.



Generación Multi-Pantalla

Crea flujos completos interconectados en un solo prompt manteniendo la coherencia de tokens de diseño.



Exportación Directa (design.md)

Exporta la 'Verdad Visual' en un documento Markdown nativamente consumible por agentes como Claude Code.

Anatomía del Export de Stitch

01 Archivo ZIP Consolidado

Descargado desde Google Stitch. Contiene todos los artefactos generados del diseño en un solo paquete.

02 design.png

La verdad visual de referencia. Claude Code la lee por visión para verificar fidelidad del resultado final.

03 index.html

Código base exportado con la estructura DOM y clases CSS del prototipo de Stitch.

04 design.md — La Joya de la Corona

Manifiesto de tokens en Markdown: paleta HEX, tipografías, espaciados y estados de componentes. El agente lo lee sin consumir tokens de visión.

Profundizando: El archivo design.md

DESIGN.MD (MANIFIESTO DE TOKENS)

Un documento Markdown estructurado para ser consumido por agentes de IA. Captura el sistema de diseño: paleta de colores (HEX), tipografías, escalas de espaciado, radios de borde y estados de componentes. Evita que la IA invente diseños en cada prompt.

Flujo de Trabajo: De Stitch a Claude Code

1

Diseño Base y Prototipado

Generar, ajustar con Voice Canvas y validar flujos en modo prototipo.

2

Exportación ZIP

Descargar el paquete consolidado de assets y extraerlo localmente.

3

Inyección de Contexto

Mover el archivo `design.md` a la raíz del repositorio de React.

4

Orquestación en Claude

Instruir a Claude Code a referenciar estrictamente `design.md` al programar.

FASE 2

CONFIGURACIÓN DEL CEREBRO AGÉNTICO

Preparando el Entorno y las Reglas de Negocio

Ajustando CLAUDE.md para nuestro Mini Jira

01

Inyección de design.md

Regla estricta: 'Referencia siempre design.md. NUNCA inventes colores o espaciados. Usa los tokens definidos ahí.'

02

Stack Tecnológico Específico

Obligar el uso de React 19 (useOptimistic), Tailwind CSS para UI y Zustand para el estado global del Kanban.

03

Reglas de Componentes

Forzar Atomic Design: Separar componentes presentacionales (TaskCard) de los contenedores lógicos (KanbanBoard).

Mejores Prácticas: Forjando CLAUDE.md

✓ QUÉ INCLUIR

- Comandos Bash específicos (NPM scripts).
- Reglas de estilo personalizadas.
- Obligatoriedad de referenciar `design.md`.
- Convenciones críticas del repositorio.

✗ QUÉ EXCLUIR

- Datos obvios deducibles del código fuente.
- Documentación extensa de APIs.
- Convenciones estándar de JavaScript/React.
- Listas exhaustivas archivo-por-archivo.



Actividad 1: Setup del Workspace

1

Proyecto Base y Blindaje

Inicializa Vite/React/Tailwind y crea el `.claudeignore`.

2

Importar Arsenal SDD

Copia `specs.md` y `backlog.md` a la raíz del proyecto.

3

Importar Assets Stitch

Mueve el archivo `design.md` y las referencias PNG a la raíz.

4

Forjar CLAUDE.md

Usa la IA para redactar el manifiesto basado en las reglas del proyecto.



Prompt: Forjando el Cerebro

""Actúa como Tech Lead. Revisa 'design.md' y 'backlog.md'. Crea el archivo 'CLAUDE.md' con las reglas globales. Regla estricta: NUNCA inventes colores, usa SOLO los definidos en 'design.md' con Tailwind. Detente cuando el archivo esté creado.""

✓ Checkpoint Actividad 1: ¿Logramos esto?



Proyecto Corriendo

Vite + React 19 + Tailwind levantado en localhost:5173 sin errores.



Arsenal SDD Copiado

specs.md, backlog.md y design.md en la raíz del proyecto.



Cerebro Forjado

CLAUDE.md generado con reglas de Stitch, Zustand y Atomic Design.

FASE 3

ESTADO, RENDIMIENTO Y LA ENTREVISTA SOCRÁTICA

Tomando Decisiones Arquitectónicas antes de Codificar

La Herramienta que Invierte el Control

ASKUSERQUESTION (TOOL DE CLAUDE CODE)

Herramienta nativa de Claude Code que pausa la ejecución y formula preguntas de opción múltiple al desarrollador antes de escribir una sola línea de código. En lugar de que la IA asuma la arquitectura correcta, **tú tomas la decisión** con plena información de los trade-offs. Es la diferencia entre un "Code Monkey" y un verdadero Consultor Técnico.

Inversión de Control: El Enfoque Socrático





Actividad 2: Entrevista Socrática — Drag & Drop

1

Invocar al Agente

Abre la terminal integrada en tu IDE y asegura que Claude esté activo con el contexto de `specs.md`.

2

Prompt Socrático sobre D&D

Pedir a la IA que, en rol de Arquitecto, evalúe las opciones de Drag & Drop para el Kanban y use `AskUserQuestion` antes de recomendar.

3

Evaluar Opciones

Analizar los pros y contras de cada alternativa (bundle size, accesibilidad, compatibilidad con React 19) y elegir.

4

Registrar la Decisión

Instalar la librería elegida y documentar la decisión en `CLAUDE.md` para que la IA la respete en todas las actividades siguientes.



Prompt: El Rol Socrático

"Asume el rol de Arquitecto Frontend. Investiga las opciones actuales de Drag & Drop para un tablero Kanban con React 19 (incluyendo dnd-kit, @hello-pangea/dnd, HTML5 nativo y cualquier alternativa relevante). Analiza bundle size, mantenimiento activo, accesibilidad y compatibilidad. Luego úsame 'AskUserQuestion' para hacerme las preguntas necesarias sobre mis restricciones y prioridades. Solo cuando tengas mis respuestas, recomienda la mejor opción e instálala."

La IA investiga primero, pregunta después, decide contigo

✓ Checkpoint Actividad 2: ¿Logramos esto?



Claude Preguntó Primero

La IA usó `AskUserQuestion` y esperó tu respuesta antes de recomendar cualquier librería.



Librería D&D Elegida

Dependencia instalada y configurada, compatible con React 19 y validada contra los requisitos del proyecto.



Decisión Documentada

La elección quedó registrada en `CLAUDE.md` para que el agente no la cuestione ni la cambie.



Fundación Lista

El mecanismo de D&D está disponible antes de construir los componentes Kanban en la siguiente actividad.

FASE 4

ATOMIC DESIGN Y OPTIMIZACIÓN DE MEMORIA

Gestionando la Ventana de Contexto del LLM

El Problema: La Amnesia Agéntica

AMNESIA AGÉNTICA (LOST IN THE MIDDLE)

Fenómeno que ocurre cuando el contexto del LLM se satura con excesivas lecturas y correcciones pasadas. La IA comienza a 'olvidar' reglas de CLAUDE.md, produciendo errores de sintaxis y alucinaciones debido a la saturación de memoria.

Cuando Claude Falla: Protocolo de Recuperación

✖ Errores Comunes

- Imports de módulos inexistentes o rutas incorrectas.
- Ignora `design.md` e inventa colores CSS propios.
- Errores de TypeScript por tipos mal inferidos.
- Rompe la estructura de Atomic Design acordada.

✔ Cómo Recuperarse

- Ejecuta `/clear` para limpiar el contexto contaminado.
- Re-inyecta `design.md` y `specs.md` en el nuevo prompt.
- Pega el mensaje de error exacto del compilador.
- Pide solo la corrección puntual, no reescribir todo.

Solución: El Patrón Document & Clear



Atomic Design Dirigido por IA

01 Átomos y Moléculas

Construir Sidebar y carcasa base aislando la lógica (Uso estricto de design.md).

02 Organismos Independientes

Construcción paralela de TaskCards aplicando sombras y estilos pre-aprobados.

03 Ensamblaje (Templates)

Unir componentes aislados en el KanbanBoard general para formar la pantalla completa.



Actividad 3.1: Home Completo por Fases Atómicas

1

Generar el Plan con `/plan`

La IA analiza el prototipo de Stitch y divide el Home en fases atómicas ordenadas: qué componente construir primero, cuáles dependen de cuáles, y qué tokens aplica cada uno. Aprobar el plan completo antes de tocar código.

2

Ejecutar Fase a Fase

La IA construye un componente por instrucción. Cada fase termina con `npm run dev` y comparación visual contra el prototipo antes de avanzar.

3

Validar y Continuar

El usuario aprueba o corrige cada fase. Solo al dar el OK se pasa a la siguiente pieza. El plan garantiza que nada queda sin construir.

4

Ensamblado Final

Todas las piezas se integran en el `Layout.tsx` del Home. Revisión global de composición, tokens y fidelidad al prototipo.



Prompt 3.1: Plan de Fases Atómicas del Home

"Tengo el prototipo en 'design.png' y los tokens en 'design.md'. Usa /plan para dividir la construcción del Home completo en fases atómicas: identifica cada componente (Sidebar, Header, área de contenido principal, KanbanBoard), define el orden de construcción de menor a mayor dependencia, y especifica qué tokens de color y tipografía aplican en cada fase. El objetivo es poder validar fase a fase hasta tener todo el Home. No escribas código hasta que apruebe el plan."

✓ Checkpoint Actividad 3.1: ¿Logramos esto?



Sidebar Visible

Sidebar.tsx renderiza correctamente con su estructura de menú en el navegador.



Layout de 2 Columnas

La estructura principal (sidebar + área de contenido) coincide con el prototipo de Stitch.



Tokens de Stitch Activos

Los colores HEX y tipografías de design.md se reflejan en las clases de Tailwind generadas.



Sin Lógica Prematura

Solo UI pura con datos mock. El componente es presentacional, sin conexiones al Store.



Actividad 3.2: Componentes Reutilizables del Kanban

1

Limpiar Contexto (/clear)

Resetear el historial de la actividad anterior para que el agente planifique sin contaminación de sesiones previas.

2

Plan enfocado en Reutilización

Usar /plan para definir componentes genéricos: TaskCard con props variantes, KanbanColumn reutilizable para cualquier estado y KanbanBoard como orquestador. Aprobar el árbol antes de codificar.

3

Ejecutar Todo de Una

Con el plan aprobado y el contexto limpio, la IA genera todos los componentes en una sola instrucción. El plan ya garantiza el orden y la atomicidad.

4

Verificar Composición

Comprobar que TaskCard funcione en cualquier KanbanColumn y que los estilos repliquen exactamente los de design.md.



Prompt 3.2: Plan de Componentes Reutilizables

"El Home ya está construido. Usa /plan para diseñar los componentes reutilizables del Kanban: define TaskCard con sus props y variantes de prioridad, KanbanColumn como columna genérica para cualquier estado, y KanbanBoard como orquestador. Incluye el árbol de dependencias y los tokens de design.md a aplicar en cada uno. Al aprobar el plan, genera todos los componentes en una sola ejecución."

Un plan atómico, una sola ejecución — reutilización desde el diseño

✓ Checkpoint Actividad 3.2: ¿Logramos esto?



Contexto Limpio

/clear ejecutado antes del prompt. El agente opera con memoria fresca.



TaskCard con Estilos Stitch

Tarjetas con sombras, bordes y colores exactos del export de Google Stitch.



KanbanBoard con 3 Columnas

Tablero visible con datos mock en "Por Hacer", "En Progreso" y "Completado".



Fidelidad Visual Confirmada

El tablero en el navegador coincide visualmente con el prototipo original de Stitch.

FASE 5

INTERACTIVIDAD Y EXPERIENCIA DE USUARIO

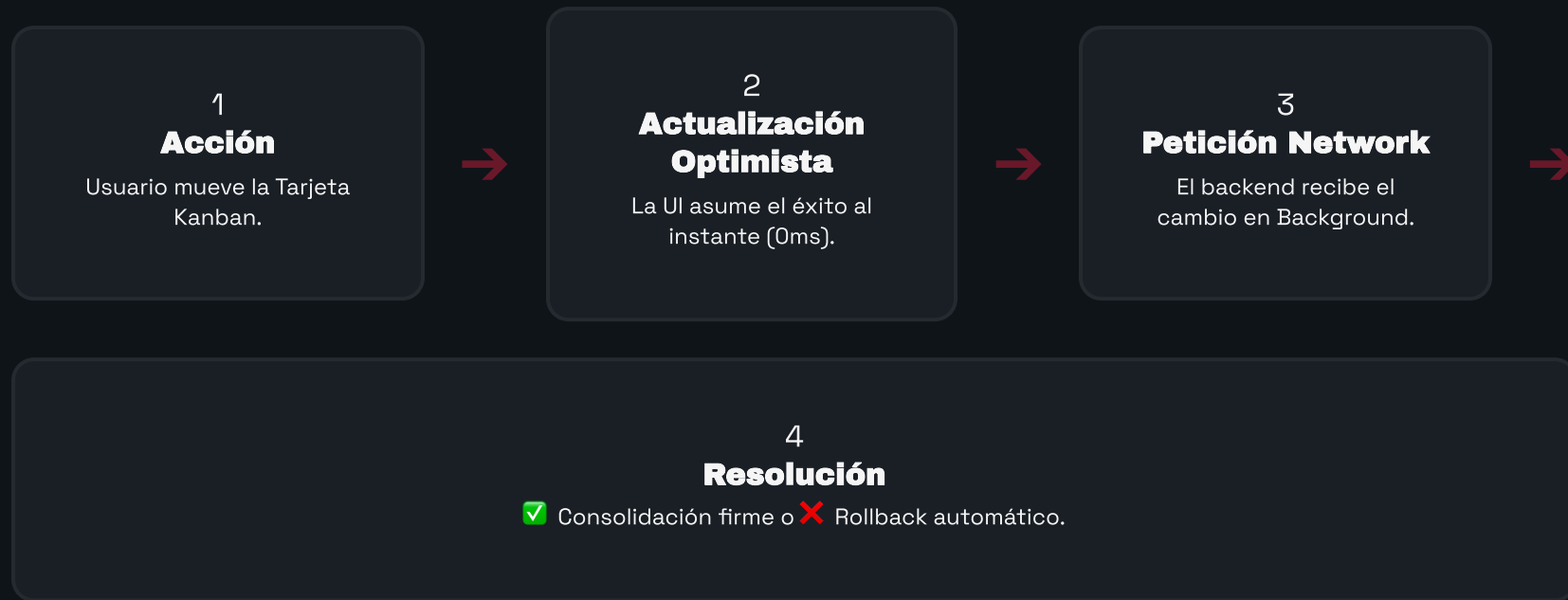
Mitigando Latencia con Datos y React 19

El Problema de la Latencia de Red

BLOQUEO SÍNCRONO SERVER-CLIENT

Si al mover una tarjeta Kanban la interfaz espera la respuesta del backend (BD) para actualizarse visualmente, se percibe un retraso (lag/spinner) que rompe la fluidez operativa y degrada la Experiencia de Usuario.

¿Qué es Optimistic UI?



React 18 vs React 19: El Hook useOptimistic

⚠️ React 18: Boilerplate Manual

- useState duplicado: uno para servidor, uno para UI optimista.
- Lógica de rollback escrita a mano con try/catch.
- Código denso y propenso a errores de sincronización.
- Difícil de mantener con múltiples acciones concurrentes.

✅ React 19: useOptimistic

- `const [optimistic, add] = useOptimistic(tasks, updater)`
- Estado visual temporal proyectado al instante (0ms).
- Rollback automático cuando la API falla.
- Elimina todo el boilerplate de sincronización manual.



Actividad 4: Inyección y Optimistic UI

1

Limpiar Contexto (/clear)

Nueva sesión: los componentes UI ya están listos. Resetear para enfocar al agente solo en la capa de integración y estado.

2

Plan de Integración con /plan

Definir cómo conectar Zustand al KanbanBoard, la firma de `useOptimistic` para el movimiento de tarjetas y la lógica de rollback. Aprobar antes de codificar.

3

Ejecutar en Una Sola Instrucción

Con el plan aprobado, la IA conecta los componentes al store, implementa `useOptimistic` y simula el backend lento con `setTimeout`.

4

Validar en el Navegador

Interactuar con el tablero y comprobar 0ms de retraso visual + rollback automático al simular un fallo.



Prompt 4: Plan de Integración

"Los componentes UI están listos. Usa /plan para diseñar la integración completa: cómo conectar KanbanBoard y TaskCard al store de Zustand, la implementación de useOptimistic de React 19 para el movimiento de tarjetas, y la lógica de rollback simulando un backend lento con setTimeout. Al aprobar el plan, ejecuta toda la integración en una sola instrucción."

✓ Checkpoint Actividad 4: ¿Logramos esto?



Movimiento a 0ms

Las tarjetas se mueven entre columnas de forma instantánea sin esperar al servidor.



Zustand Conectado

El KanbanBoard consume los selectores del Store y actualiza solo lo necesario.



Rollback Probado

Al forzar el fallo del `setTimeout`, la tarjeta regresa a su columna original automáticamente.



Mini Jira Completo

Aplicación funcional con diseño Stitch, estado Zustand y UX de grado empresarial.

Resumen: El Ingeniero Frontend 2026

Orquestador, no mecanógrafo

El valor reside en dominar la arquitectura, el rendimiento y la orquestación de agentes de IA mediante contexto estricto.

10x Velocidad

Conectando el ZIP de Stitch con Claude Code CLI.

Cero Caos

Eliminación del Vibe Coding a favor de la ejecución Determinista (SDD).

Rendimiento UX

Uso mandatorio de Zustand y useOptimistic para eliminar latencias de red.

Q & A

PREGUNTAS Y RESPUESTAS

¡GRACIAS!